

* Function as parameter

⇒ In python functions are 1st class objects

We can assign them to variables

Ex. `def square(x):`

`return x*x`

`def cube(x):`

`return x*x*x`

`def apply_fun(fun, num):`

`return fun(num)`

`apply_fun(square, 7)`

o/p

49

`apply_fun(cube, 5)`

o/p

125

* Nested Functions:

⇒ A nested function is a function defined inside another function.

- It helps in encapsulation and modularity.

Ex.

```
def outer_fun(name):  
    def inner_fun():  
        return f"hello, {name}"  
    return inner_fun()  
outer_fun("Kiran")
```

o/p 'hello, kiran'

Key points

The inner function can be access variables from the outer functions (called closure), nested functions are often used in decorators and encapsulation.

* Recursion Functions

A recursive function that calls itself until the condition is satisfied.

- Base case: Condition to stop recursion.
- Recursive case: Functions calls itself with modified input.

EX: `def factorial(n):`
 `if n == 1:`
 `return 1`
 `else:`
 `return n * factorial(n-1)`
`factorial(6)`

o/p 720

EX1: Fibonacci Series = 0, 1, 1, 2, 3, 5, 8, 13

`a = 0`
`b = 1`
`for i in range(1, 10, 1):`
 `print(a, end=" ")`
 `a, b = b, a+b`

o/p 0 1 1 2 3 5 8 13 21

③ lambda function

⇒ It is a small anonymous (nameless) function in Python.

- Defined using the keyword `lambda` instead of `def`.
- Can take any number of arguments but must contain only one expression.

- Expression is automatically returned (no need to use return)

Syntax

lambda arguments : expression

uses of lambda

- useful for short, simple operation
- saves code (one-line)
- often used with map(), filter(), or a inline.

Ex: $Sq = \text{lambda } x : x * x$
 $Sq(5)$

o/p 25

Ex1: $\text{add} = \text{lambda } a, b : a + b$
 $\text{add}(3, 8)$

o/p 11